

COMPSCI 701:
Creating Maintainable Software
Lecture 4.1: Programming Conventions

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

- Admin
 - ?
- Assignment 2
- Programming conventions — what they look like and why we care

- Agenda
- **Assignment 2**
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Assignment 2

- “Alter” your Assignment 1 submission to allow, simultaneously, the change cases:
 - Sowing seeds in a **anti-clockwise** direction.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| P2 | 6[ 4] | 5[ 4] | 4[ 4] | 3[ 4] | 2[ 4] | 1[ 4] | 0 |
|    |-----+-----+-----+-----+-----+-----+-----+
| 0 | 1[ 4] | 2[ 4] | 3[ 4] | 4[ 4] | 5[ 4] | 6[ 4] | P1 |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

- The board has **7 houses per player** that are initialised with **5 seeds**.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| 0 | 1[ 5] | 2[ 5] | 3[ 5] | 4[ 5] | 5[ 5] | 6[ 5] | 7[ 5] | P2 |
|    |-----+-----+-----+-----+-----+-----+-----+
| P1 | 7[ 5] | 6[ 5] | 5[ 5] | 4[ 5] | 3[ 5] | 2[ 5] | 1[ 5] | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

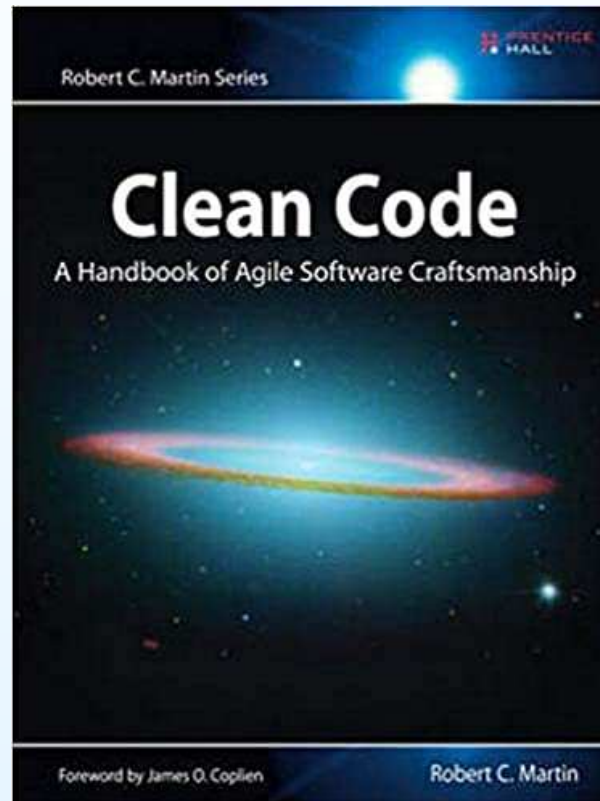
- All combinations must be supported (including what was used for Assignment 1)


```
public void play(IO io, boolean clockwise, boolean sevenfive, boolean p3) {
    ...
}
```
- Write a “report” on what you had to do to implement the change cases and assess the alterability of your Assignment 1 submission.
- Accept Invite for GitHub Classroom assignment repo on Canvas
- Due midnight Friday of Week 6

Previously in COMPSCI701: Clean Code

- Agenda
- Assignment 2
- **Previously**
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

- Robert C. Martin *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson (2008)



- code is read more than it is written $\Rightarrow$ write the code for the readers
- Much of the advice is about how to write code that can be understood

- Agenda
- Assignment 2
- Previously
- **Programming Conventions**
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Programming Conventions

- Much advice, many suggestions, many rules, have been proposed to determine what “good code is”
- Presented as “programming conventions”, “coding conventions”, “code style”, “style guidelines”, and variations
- Based on hard-won experience as to what **is perceived** to affect (mostly) maintainability
- Mostly refers to **comprehensibility**, or occasionally **alterability**
- Presented in books (e.g. “Clean Code” by Martin, “Code Complete” by McConnell), documents (e.g. PEP8 for Python¹, Google style guide for Java²) Oracle style guide for Java³
- Many tools (generally called “linters”) for checking that guidelines have been met (e.g. **PMD**, Checkstyle, FindBugs, Lint, Pylint) or provide “measurements” that may indicate when guidelines have not been met (e.g. SonarQube, Understand).
- Many IDEs will provide some checking of guidelines
- The sheer number of sources that say some version of “you must follow program conventions to get good code” suggest there is some truth to this advice — **Everyone agrees everyone should have and follow a “style guide”**

¹<https://peps.python.org/pep-0008>

²<https://google.github.io/styleguide/javaguide.html>

³<https://www.oracle.com/java/technologies/javase/codeconventions-introduction.html>

- Agenda
- Assignment 2
- Previously
- **Programming Conventions**
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

But!

- Different conventions have significant differences to each other
- Little in the way of scientific evidence, even for the ones we believe
- Some conventions seem arbitrary choices
- Some conventions are inconsistent with other guidelines
- Some conventions seem to be just plain wrong
- A number of conventions use of thresholds, which creates ambiguity
- A number of requires subjective decisions
- Often no **rationale**

Example Advice (Python)

- Do not include 'l', 'I' or 'O' in variable names. They are easy to confuse with numbers. (Řečtáčková et al.⁴)
- Limit all lines to a maximum of **79 characters**. (PEP8)

OR

Limit lines to **100 characters**. (Řečtáčková et al.)

- Avoid **extraneous** whitespace in the following situations: (long list omitted) (PEP8)
- Block comments should start **at least 2 spaces** away from the code with the comment character #, followed by at least one space before the text of the comment itself.
- While **sometimes** it's okay to put an if/for/while with a small body on the same line, never do this for multi-clause statements. Also avoid folding such long lines!
- While **sometimes** it's okay to put an if/for/while with a small body on the same line, never do this for multi-clause statements. Also avoid folding such long lines!
- **Two** blank lines between top-level definitions, be they function or class definitions (Google)
- **Python coders from non-English speaking countries: please write your comments in English, unless you are 120% sure that the code will never be read by people who don't speak your language** (PEP8)
- If the branches of a conditional contain the same code, reorganise to have a single branch with a **more complex** condition. (Řečtáčková et al.)
- If a function contains a local variable that is used only in a return statement, remove the variable and return the value. (Řečtáčková et al.)

⁴Řečtáčková et al. et al. "Catalog of Code Quality Defects in Introductory Programming". ITiCSE 2024. <https://doi.org/10.1145/3649217.3653638>

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- **Examples**
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Example Rules (PMD, Java)

- ControlStatementBraces, FieldDeclarationsShouldBeAtStartOfClass, SimplifyBooleanExpressions, SimplifyBooleanReturns, AvoidReassigningLoopVariables, AvoidReassigningParameters, EmptyCatchBlock, AvoidCatchingGenericException, AbstractClassWithoutAnyMethod, CyclomaticComplexity, AvoidDeeplyNestedIfStmts, PackageCase, ClassNamingConventions, MethodNamingConventions, FormalParameterNamingConventions, LocalVariableNamingConventions, FieldNamingConventions, DefaultLabelNotLastInSwitch, UnusedAssignment, UnusedLocalVariable, UnusedPrivateField, UnusedPrivateMethod, UnnecessaryReturn, UnnecessaryImport, UnnecessaryCast, ShortMethodName, ShortVariable, ShortClassName, MethodReturnsInternalArray, ReplaceHashtableWithMap, ReplaceVectorWithList, fieldvisibility, CouplingBetweenObjects, DataClass, GodClass, LooseCoupling, ConstantsInInterface, NcssCount, TooManyFields, TooManyMethods, ExcessiveParameterList
- (Rules used in Assignment 1)

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- **Issues**
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Guideline-specific rules

- It is not hard to find rules/advice in one set of conventions that do not appear in others
 - E.g. Apache “Code that parses special input strings (e.g, comma delimited) should provide examples of valid strings in comments”
<https://cwiki.apache.org/confluence/display/CLOUDSTACK/Coding+conventions>
 - E.g. Google 4.8.3.2 “No C-style array declarations”
<https://google.github.io/styleguide/javaguide.html#s4.8.3-arrays>
- Are such rules/advice somehow less important than those rules that appear in all sets of conventions?

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- **Issues**
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Arbitrary

- For some rules/advice, it is unclear what issue is being addressed, and so they seem arbitrary
 - E.g. (PEP8) Avoid extraneous whitespace in the following situations:
... Between a trailing comma and a following close parenthesis:

```
# Correct:
foo = (0,)

# Wrong:
bar = (0, )
```

If knowing that a trailing comma exists (as another PEP8 rule suggests), then surely adding the space makes it easier to see?

- What is the reason for having these rules? What goes wrong if they are not obeyed?

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- **Issues**
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Inconsistent

- Often, rules/advice given in one set of conventions is different to another.
 - Limit all lines to a maximum of **79 characters**. (PEP8)
 - **OR**
 - Limit lines to **100 characters**. (Řechtáčková et al.)
 - Where (and whether) to put braces
(https://en.wikipedia.org/wiki/Indentation_style lists 9 different conventions)
 - How to format multiple-word identifiers
([https://en.wikipedia.org/wiki/Naming_convention_\(programming\)](https://en.wikipedia.org/wiki/Naming_convention_(programming)) lists 11 choices)
- How do we know which are “correct”? If it doesn’t matter, why follow any rules?

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Not obviously correct

- Some rules/advice seem (or actually are) wrong
 - AbstractClassWithoutAbstractMethod The abstract class does not contain any abstract methods. (PMD)
 - Python coders from non-English speaking countries: please write your comments in English, unless you are 120% sure that the code will never be read by people who don't speak your language (PEP8)
- Without knowing why these rules have been proposed, it is tempting to just ignore them (but do we dare?)

Thresholds

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

- The effectiveness of any rule that is based on the use of a threshold depends on the choice of value for the threshold:
 - too “low” causes false positives that result in developers ignoring all reports
 - too “high” and false negatives that result high potential problems
 - generally — how seriously should violations of such rules be taken?
- E.g.
 - PMD Default for ShortVariable:

```
<rule ref="category/java/codestyle.xml/ShortVariable">
  <properties>
    <property name="minimum" value="3" />
  </properties>
</rule>
```

PMD report

Problems found

#	File	Line	Problem
1	src/kalah/Kalah.java	15	Avoid variables with short names like io

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- **Clean Code**
- Class Exercise
- Conclusion
- Key Points

Martin on “Formatting”

- Many programming conventions focus on “formatting” (more generally, how code is **presented**)
- Martin on Formatting (Chapter 5)

We want them to perceive that professionals have been at work. If instead they see a scrambled mass of code that looks like it was written by a bevy of drunken sailors, then they are likely to conclude that the **same inattention to detail pervades every other aspect** of the project.

First of all, let's be clear. Code formatting is *important*. It is too important to ignore and it is too important to treat religiously. Code formatting is about communication, and **communication is the professional developer's first order of business**.

- these statements do not explain how programming conventions (or even just formatting) improves comprehensibility (or maintainability)

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- **Clean Code**
- Class Exercise
- Conclusion
- Key Points

Martin on Presentation

- Vertical presentation
 - File size (Martin: about 200 LOC, max 500)
 - Ordering (Martin: Functions call functions declared below, Concepts closely related should be kept close together)
 - Grouping (Martin: Blank lines provide visual cues separating “concepts”)
- Horizontal presentation
 - Indentation
 - Line width (Martin 45 characters)
 - Use whitespace to separate (or not) to indicate how close the relationship

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- **Class Exercise**
- Conclusion
- Key Points

Class Exercise

- Form groups
- Provide answers to:
 1. Choose one of maximum line widths of **79** and **100** characters, and justify your choice — using the Program Comprehension Model. That is, explain why your choice is better than the other option.

Hint: Consider the efficiency and/or effectiveness of the access of the Assimilation Process to the External Representation
 2. PMD has default values of 5 for ShortClassName (i.e. classes with names shorter than 5 characters are reported as violations) and 3 for ShortMethodName. Pick one of these rules and:
 - (a) Find an example name that is a **reasonable** name, but violates your chosen rule
 - (b) Justify why your choice of name is reasonable using the Program Comprehension Model
 - (c) (If time) Should the default value for the PMD rule be changed?

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- **Conclusion**
- Key Points

Should there be Programming Conventions?

- Everyone (including us) agrees that **some** conventions should be adopted and followed, at least within an organisation
- Expect that any organisation you join will have a standard you should follow (and if not ask why not!)
- But, **ensure you understand what benefit you expect from following the conventions** (i.e. the rationale)
- (Note: Not all conventions are intended to help with comprehensibility, alterability, or testability)
- **Unfortunately many conventions are given without explanation**

- Agenda
- Assignment 2
- Previously
- Programming Conventions
- Examples
- Issues
- Clean Code
- Class Exercise
- Conclusion
- Key Points

Key Points

- Following programming conventions is generally believed to lead to more maintainable code
- Often conventions seem arbitrary, are inconsistent, ambiguous, subjective, and even just plain wrong
- In order to evaluate the validity of any convention, we need to know the **rationale** for why it should be followed — very often this is not provided